Lab 10: File Operations

Computer Programming () |
Duration: 2 hours (in-lab) | **Topic:** Reading and writing text files with `FILE *`

Objectives

- Open and close text files with `fopen`/`fclose`; check for failure.
- Read with `fscanf`, `fgets`, and `fgetc`; write with `fprintf`, `fputs`, `fputc`.
- Detect end-of-file correctly using the function return value, not `feof`.
- Combine file I/O with previous skills (loops, arrays, structs) to process data sets.

Quick Reference

A file handle is a pointer. `fopen` returns a `FILE *` – a pointer to an internal bookkeeping struct managed by the standard library. You never dereference it yourself; you just pass it to other `f*` functions. If `fopen` fails it returns `NULL` – *always* check.

Open / close.

```c
FILE *f = fopen("input.txt", "r");
if (f == NULL) { perror("fopen"); return 1; }
/* ... use f ... */
fclose(f);
```

Modes. `"r"` read, `"w"` write (truncates), `"a"` append, add `"+"` for read+write.

Read. `fscanf(f, "%d", &x)` returns the number of items successfully read, or `EOF`. `fgets(line, sizeof line, f)` returns `NULL` at end of file or on error.

Idiomatic read loop.

```c
int x;
while (fscanf(f, "%d", &x) == 1) {
    /* process x */
}
```

Write. `fprintf(f, "%d %s\n", id, name);`

In-Lab Exercises (target: 2 hours)

Before you start: create a sample ***`numbers.txt`*** *(one integer per line) and a* ***`lines.txt`*** *(any text) in your working directory.*

In-Lab Exercise 1: Copy a Text File *(25 min)*

Write a program that reads `lines.txt` line-by-line with `fgets` and writes each line to `copy.txt`. Verify by opening `copy.txt` in your editor. Handle the case where the input file cannot be opened.

In-Lab Exercise 2: Sum and Average from File *(30 min)*

Read all integers from `numbers.txt` (unknown count) with `fscanf`. Print the count, sum, and average. Use the return value of `fscanf` to detect end of file – do not call `feof` as the loop condition.

In-Lab Exercise 3: Word and Line Count *(35 min)*

Read `lines.txt` character by character with `fgetc`. Count and print: the number of characters, the number of words (whitespace-separated), and the number of lines.

In-Lab Exercise 4: Students Report *(30 min)*

Create `students.txt` with lines of the form `<id> <name> <score>`, e.g. `1023 Bob 87.5`. Read the records into an array of structs (reuse the `Student` struct from Lab 8) and write `report.txt` with the same records sorted by score in descending order.

Take-Home (Paper Work). Solve the following on paper without using a computer or compiler. Hand-write your answers; submit at the start of the next lab. Goal: prepare you to dry-code during the written exam.

Trace & Predict 1: Read loop terminates correctly

The file `nums.txt` contains: `10 20 30 40`. What does this print?

```c
#include <stdio.h>
int main(void) {
    FILE *f = fopen("nums.txt", "r");
    int x, n = 0, s = 0;
    while (fscanf(f, "%d", &x) == 1) { s += x; n++; }
    fclose(f);
    printf("%d %d\n", n, s);
    return 0;
}
```

Trace & Predict 2: `"w"` truncates

If `out.txt` already contains `"old"`, what does `out.txt` contain after this program runs?

```c
#include <stdio.h>
int main(void) {
    FILE *f = fopen("out.txt", "w");
    fprintf(f, "new\n");
    fclose(f);
    return 0;
}
```

Find the Bug 1: Missing check, wrong mode

List the bugs and rewrite:

```c
#include <stdio.h>
int main(void) {
    FILE *f = fopen("data.txt", "w");    /* we only want to read */
    int x;
    while (!feof(f)) {                   /* anti-pattern */
        fscanf(f, "%d", &x);
```

```
        printf("%d\n", x);
    }
    /* fclose forgotten */
    return 0;
}
```

Write Code on Paper 1: Append a line

Write a complete program that prompts the user for a single line of text and appends it (followed by a newline) to `log.txt`. Create the file if it does not exist, but never erase existing content.

Write Code on Paper 2: Merge two sorted files

Two files `a.txt` and `b.txt` each contain an ascending sequence of integers. Write a complete program that produces `merged.txt` containing the combined sequence, still in ascending order, by reading one number at a time from each input.

End of the lab series. Notice how every lab from Lab 7 onward has used pointers: out-parameters (Lab 7), `char *` string traversal (Lab 8), `struct *` with `->` (Lab 9), and `FILE *` here. Keep your in-lab source files and paper-work together as a portfolio for revision before the final exam.